

实验四 数组和串实验

数组基础练习一

【问题描述】

题目要求设数组 $R[0..n-1]$ 中有多个 0 元素，设计一个算法把所有非 0 元素依次移动到数组前端。移动后，非 0 元素仍保持原有先后顺序，0 元素集中留在数组后部。

本实验在数组/ex_1 中实现 `move_non_zero_front(data: &mut [i32])`。主程序用示例数组 `[0, 3, 0, 8, 0, 2, 5]` 演示移动前后的变化，测试代码验证典型情况、全 0 和无 0 等边界输入。

【基本思路】

算法使用两个下标完成原地调整。 i 表示下一个非 0 元素应该放置的位置， j 从左到右遍历整个数组。当 `data[j]` 不为 0 时，若 i 与 j 不相等，则交换 `data[i]` 和 `data[j]`，随后 i 向后移动。

这种做法不需要另开数组，所有操作都在原数组上完成。每个元素最多被检查一次，因此时间复杂度为 $O(n)$ ，额外空间复杂度为 $O(1)$ 。

`move_non_zero_front(data)`

`i <- 0`

`for j from 0 to data.len-1`

`if data[j] != 0`

`if i != j then 交换 data[i] 和 data[j]`

`i <- i + 1`

函数调用关系可表示为：`main()` -> 构造示例数组 -> `move_non_zero_front()` -> 输出移动后的数组。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 数组/ex_1/src/lib.rs，主程序位于 数组/ex_1/src/main.rs，测试代码位于 数组/ex_1/tests/test_fn.rs。核心函数只接收可变切片，不依赖文件或全局变量。

测试数据覆盖混合 0 与非 0、全 0、无 0 三类情况，重点验证非 0 元素的相对顺序是否保持，以及边界输入是否不会引发越界。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	<code>[0, 3, 0, 8, 0, 2, 5]</code>	移动后为 <code>[3, 8, 2, 5, 0, 0, 0]</code> ；非 0 元素顺序保持不变。
2	<code>[0, 0, 0]</code>	移动后仍为 <code>[0, 0, 0]</code> ；说明全 0 输入能正确处理。
3	<code>[1, 2, 3]</code>	移动后仍为 <code>[1, 2, 3]</code> ；说明无 0 输入不会被误交换。
4	空数组 <code>[]</code>	循环不执行，数组保持为空；验证边界条件。

运行 cargo test 后，本题 2 个单元测试全部通过；运行 cargo run 后，程序输出移动后数组 [3, 8, 2, 5, 0, 0, 0]，与题目要求一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于正确维护 i 和 j 两个下标，使非 0 元素稳定地移动到前部；难点在于交换后不能破坏未处理区域，也不能因为数组为空或全 0 而产生越界。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：算法简洁、原地完成、只进行一次线性扫描。可以改进的方向包括：把函数改为泛型并接收一个“是否为空值”的判断函数，以支持更多类型的数据移动。

还有哪些实现方法？

还可以先统计非 0 元素并写回数组前部，再把剩余位置置 0；也可以使用语言库中的稳定分区思想实现，但当前双下标交换写法更贴近题目提示。

【问题描述】

题目要求设计一个算法，将 $A[0..n-1]$ 中所有奇数移到偶数之前，并且不另增加存储空间，时间复杂度为 $O(n)$ 。题目没有要求保持奇数或偶数内部的原有顺序。

本实验在数组/ex_2 中实现 `odd_before_even(data: &mut [i32])`。主程序使用 `[2, 9, 4, 7, 6, 3, 8, 1]` 演示调整过程，测试代码验证奇偶分区结果和短数组边界情况。

【基本思路】

算法采用左右双指针。 i 从左向右寻找位于左侧的偶数， j 从右向左寻找位于右侧的奇数，当 $i < j$ 时交换两个元素。交换后继续向中间推进，直到 i 与 j 相遇或交错。

由于每次移动都至少推进一个指针，整个数组最多被扫描一遍。算法只使用常数个下标变量，满足不增加额外存储空间和 $O(n)$ 时间复杂度的要求。

```
odd_before_even(data)
  if data.len < 2 return
  i <- 0, j <- data.len - 1
  while i < j
    while i < j 且 data[i] 为奇数, i <- i + 1
    while i < j 且 data[j] 为偶数, j <- j - 1
    if i < j, 则交换 data[i] 和 data[j]
  函数调用关系可表示为: main() -> 构造示例数组 -> odd_before_even() -> 输出调整后的数组。
```

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 数组/ex_2/src/lib.rs, 主程序位于 数组/ex_2/src/main.rs, 测试代码位于 数组/ex_2/tests/test_fn.rs。函数直接修改传入的可变切片。

测试重点不是固定某一个唯一排列，而是检查所有奇数是否都出现在所有偶数之前，同时验证空数组、单元素数组、全奇数和全偶数输入。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	[2, 9, 4, 7, 6, 3, 8, 1]	调整后为 [1, 9, 3, 7, 6, 4, 8, 2]; 奇数均在偶数之前。
2	[1, 3, 5]	数组保持为全奇数序列; 左指针会直接越过所有元素。
3	[2, 4, 6]	数组保持为全偶数序列; 右指针会直接越过所有元素。
4	[] 和 [2]	函数直接返回, 不发生越界访问。

运行 `cargo test` 后，本题 2 个单元测试全部通过；运行 `cargo run` 后，程序输出调整后数组 `[1, 9, 3, 7, 6, 4, 8, 2]`，满足“奇数在偶数之前”的要求。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于理解左右指针分别寻找错误位置的元素；难点在于指针移动条件必须

始终带有 $i < j$ 判断，避免 `usize` 下标在 j 递减时出现越界。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：原地交换、空间开销小、时间复杂度符合要求。可以改进的方向包括：如果保持奇数和偶数内部相对顺序，可改用稳定移动算法，但那通常需要更多移动次数。

还有哪些实现方法？

还可以从左到右扫描，遇到奇数就插入到前方奇数区末尾；也可以新建两个数组分别保存奇数和偶数后再合并，但这不符合题目“不另增加存储空间”的要求。

【问题描述】

题目要求以三元组表表示法表示稀疏矩阵，实现两个矩阵相加、相减和相乘的运算，并把运算结果以阵列形式列出。稀疏矩阵中只保存非零元素的行号、列号和值。

本实验在数组/ex_3 中定义 Triple 和 SparseMatrix。SparseMatrix 内部保存矩阵行数、列数和三元组数组，并提供 from_array、to_array、add、sub、mul 等操作。

【基本思路】

建立矩阵时先扫描二维数组，遇到非零元素就记录为一个三元组。输出阵列时先构造全 0 二维数组，再把三元组中的值填回对应位置。

加法和减法要求两个矩阵行列数相同；乘法要求左矩阵列数等于右矩阵行数。为了保持代码对 Rust 初学者友好，本实验运算时先转为普通二维数组，再使用教材中的三重循环完成计算，最后重新压缩为三元组表。

SparseMatrix::from_array(array)

遍历每个位置 array[i][j]

若值不为 0，则保存 Triple(row=i, col=j, value=array[i][j])

add/sub

若行列不一致返回 None

对每个位置做对应加减

mul

若 self.cols != other.rows 返回 None

result[i][j] <- sum(a[i][k] * b[k][j])

函数调用关系可表示为：main() -> SparseMatrix::from_array() -> add() / sub() / mul() -> to_array() -> 输出阵列形式结果。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

本部分代码位于 数组/ex_3/src/lib.rs，主程序位于 数组/ex_3/src/main.rs，测试代码位于 数组/ex_3/tests/test_fn.rs。主程序演示矩阵 A、B 的加减和矩阵 A、C 的乘法。

测试数据覆盖三元组建立、阵列还原、矩阵加法、矩阵减法、矩阵乘法以及维度不匹配返回 None 的情况。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	A=[[1,0,2],[0,3,0]]	三元组为 (0,0,1)、(0,2,2)、(1,1,3), 还原阵列与原矩阵一致。
2	A + B, B=[[0,4,0],[5,0,6]]	结果为 [[1,4,2],[5,3,6]], 非零元素正确合并。
3	A - B	结果为 [[1,-4,2],[-5,3,-6]], 负数结果可正常保存为三元组。
4	A * C, C=[[1,2],[0,3],[4,0]]	结果为 [[9,2],[0,9]]; 维度满足 2x3 与 3x2 相乘。

运行 cargo test 后, 本题 4 个单元测试全部通过; 运行 cargo run 后, 程序输出 A+B、A-B、A*C 的阵列结果分别为 [[1,4,2],[5,3,6]]、[[1,-4,2],[-5,3,-6]]、[[9,2],[0,9]]。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于三元组表与阵列形式之间的相互转换; 难点在于矩阵乘法的维度判断和下标循环, 任何一个行列关系写错都会导致结果错误。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 结构清楚, 三元组表示和矩阵运算接口分离, 便于测试。可以改进的方向包括: 直接在三元组表上合并同位置元素, 从而减少转为二维数组带来的额外空间。还有哪些实现方法?

还可以把三元组按行列排序后使用归并思想完成加减; 乘法也可以先对右矩阵转置, 再按行列非零项相乘, 这样更能体现稀疏矩阵节省计算量的优势。

串基础练习二

【问题描述】

题目要求采用定长顺序存储结构存储串, 并设计一个算法把串中所有字符倒过来重新排列。实际 Rust 实现中使用 Vec<char> 模拟顺序存储结构, 便于按下标交换字符。

本实验在串/ex_1 中实现 reverse_string(text: &str) -> String。主程序以 datastructure 为示例, 输出逆置后的字符串。

【基本思路】

先把字符串按字符收集到 Vec<char> 中, 再使用 i 和 j 分别指向首尾字符。只要 i < j, 就交换两个位置的字符, 然后 i 后移、j 前移, 直到两个指针相遇。

该算法与数组逆置完全一致, 只需要常数个下标变量。因为 Rust 的字符串按字节存储, 直接用字节下标处理中文等字符不安全, 所以这里先转为 char 数组再处理。

reverse_string(text)

chars <- text 中所有字符

if chars.len < 2 return text

i <- 0, j <- chars.len - 1

while i < j

交换 chars[i] 和 chars[j]

i <- i + 1, j <- j - 1

返回 chars 组成的新字符串

函数调用关系可表示为: `main() -> reverse_string() -> 输出逆置结果`。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 串/ex_1/src/lib.rs, 主程序位于 串/ex_1/src/main.rs, 测试代码位于 串/ex_1/tests/test_fn.rs。函数返回新的 String, 不修改原始字符串。

测试数据覆盖普通字符串、空字符串和单字符字符串, 重点验证首尾交换循环在短串上能够正确停止。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	datastructure	逆置后为 erutcurtsatad; 与主程序输出一致。
2	a	逆置后仍为 a; 单字符无需交换。
3	空串 ""	逆置后仍为空串; 循环不会执行。
4	abccba	逆置后仍为 abccba; 说明对回文串处理正确。

运行 cargo test 后, 本题 2 个单元测试全部通过; 运行 cargo run 后, 程序输出“逆置: erutcurtsatad”, 与预期一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把串看作顺序存储的字符表来处理; 难点在于 Rust 字符串不能直接按字符下标修改, 需要先转换为 Vec<char>。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 算法直观, 和教材中定长顺序串的下标交换思想一致。可以改进的方向包括: 将函数改为直接修改一个可变字符数组, 以更贴近“定长顺序存储结构”的原题设定。

还有哪些实现方法?

还可以从后向前遍历原串并逐个 push 到新字符串中; 也可以调用标准库的 chars().rev(), 但手写首尾交换更能体现本题算法思想。

串基础练习四

【问题描述】

题目说明如果第二个字符串以第一个字符串作为开始字符串,就称第一个字符串是第二个字符串的前缀。例如 Wonder 是 Wonderful 的前缀。要求编程判断某字符串是否是另一个字符串的前缀。

本实验在串/ex_2 中实现 is_prefix(prefix: &str, text: &str) -> bool。主程序使用 Wonder 和 Wonderful 作为示例。

【基本思路】

先把两个字符串都转换为 Vec<char>。若前缀串长度大于目标串长度,则一定不是前缀;否则从下标 0 开始逐个比较两个字符,只要出现不相等就返回 false,全部相等则返回 true。

该算法只比较前缀长度范围内的字符,时间复杂度为 $O(m)$,其中 m 为待判断前缀串长度。

```
is_prefix(prefix, text)
  p <- prefix 的字符数组
  t <- text 的字符数组
  if p.len > t.len return false
  for i from 0 to p.len-1
    if p[i] != t[i] return false
  return true
```

函数调用关系可表示为: main() -> is_prefix() -> 输出 true 或 false。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 串/ex_2/src/lib.rs, 主程序位于 串/ex_2/src/main.rs, 测试代码位于 串/ex_2/tests/test_fn.rs。函数只返回判断结果,不负责输入输出。

测试数据覆盖正确前缀、空前缀、首部字符不完全相同以及前缀串比目标串更长的情况。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	Wonder, Wonderful	返回 true; Wonderful 以 Wonder 开始。
2	空串, abc	返回 true; 空串可视为任意字符串的前缀。
3	Word, Wonderful	返回 false; 第 4 个字符不同。
4	Wonderful, Wonder	返回 false; 前缀串长度超过目标串。

运行 cargo test 后, 本题 1 个单元测试通过; 运行 cargo run 后, 程序输出“Wonder 是否为 Wonderful 的前缀: true”, 与题意一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于明确“前缀”必须从目标串第一个字符开始匹配; 难点较小, 主要是要先判断长度, 避免访问目标串不存在的位置。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：判断过程清晰，边界条件完整。可以改进的方向包括：增加大小写忽略选项，或在主程序中支持从键盘输入两个字符串。

还有哪些实现方法？

还可以直接调用标准库 `starts_with` 方法完成判断；也可以通过求子串 `text[0..prefix.len]` 后比较，但手写逐字符比较更适合作为本题练习。

串进阶练习二

【问题描述】

题目要求假设以定长顺序存储结构表示串，设计 `maxcommonstr` 算法，求串 `s` 和串 `t` 的一个最长公共子串，并带出最长长度 `maxlen` 以及该子串在 `s`、`t` 中的起始位置 `pos1`、`pos2`。

本实验在串/ex_3 中实现 `max_common_substring(s: &str, t: &str) -> CommonResult`。
`CommonResult` 保存 `max_len`、`pos1`、`pos2` 和公共子串本身 `text`。

【基本思路】

算法枚举 `s` 中每一个起始位置 `i` 和 `t` 中每一个起始位置 `j`，然后从这两个位置开始向后逐字符比较，统计连续相同的长度 `len`。当 `len` 大于当前最大长度时，更新 `max_len`、`pos1` 和 `pos2`。

该方法直接、易懂，适合初学者验证最长公共子串定义。虽然时间复杂度较高，但对实验中的小规模字符串足够清晰可靠。

```
max_common_substring(s, t)
```

```
    best_len <- 0, best_pos1 <- 0, best_pos2 <- 0
```

```
    for i from 0 to s.len-1
```

```
        for j from 0 to t.len-1
```

```
            len <- 0
```

```
            while s[i+len] == t[j+len]
```

```
                len <- len + 1
```

```
            if len > best_len, 则更新 best_len、best_pos1、best_pos2
```

函数调用关系可表示为：`main()` -> `max_common_substring()` -> 输出公共子串、长度和两个起始位置。

【实验结果及分析】

1. 实验结果:

1.1 实验过程描述

本部分代码位于 串/ex_3/src/lib.rs，主程序位于 串/ex_3/src/main.rs，测试代码位于 串/ex_3/tests/test_fn.rs。程序用 `student` 和 `deskstudy` 演示最长公共子串计算。

测试数据覆盖存在公共子串和完全没有公共子串两种情况，重点验证 `max_len`、`pos1`、`pos2` 和 `text` 是否一致。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	s=student, t=deskstudy	最长公共子串为 stud, 长度 4, pos1=0, pos2=4。
2	s=abc, t=XYZ	无公共字符, max_len=0, pos1=0, pos2=0, text 为空。
3	s=ababc, t=babcd	最长公共子串可为 babc, 算法会记录第一次发现的最长匹配。
4	任一字符串为空	外层循环不执行, 最长长度保持为 0。

运行 cargo test 后, 本题 2 个单元测试全部通过; 运行 cargo run 后, 程序输出最长公共子串 stud、长度 4、s 中起始位置 0、t 中起始位置 4, 结果正确。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于区分“最长公共子串”和“最长公共子序列”, 本题要求连续匹配; 难点在于更新最大值时要同时记录两个串中的起始位置。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是: 枚举逻辑直接, 返回结果完整, 便于对照题目带出参数。可以改进的方向包括: 使用动态规划表把重复比较结果保存下来, 提高较长字符串上的效率。

还有哪些实现方法?

还可以从最长长度开始枚举所有子串, 找到第一个同时出现于两个字符串中的子串即停止; 也可以使用后缀数组等高级方法, 但这些方法对本实验而言不够直观。

串进阶练习三

【问题描述】

题目要求采用定长顺序存储结构存储串，并利用串的基本运算编写算法：删除串 s_1 中所有 s_2 子串。若删除一次后形成新的匹配，也应继续删除，直到 s_1 中不再含有 s_2 。

本实验在串 `/ex_4` 中实现 `delete_substring(s1: &str, s2: &str) -> String`，并编写 `find_substring` 辅助函数查找当前第一次出现的位置。

【基本思路】

若 s_2 为空串，则直接返回原串，避免无限循环。否则把 s_1 和 s_2 转换为字符数组，反复查找 s_2 在当前 s_1 中第一次出现的位置；如果找到，就从该位置连续删除 $s_2.len$ 个字符；如果找不到，循环结束。

这种方法把“查找子串”和“删除子串”分开实现，逻辑接近教材中的串基本运算，适合逐步调试。

`delete_substring(s1, s2)`

if s_2 为空 return s_1

$data \leftarrow s_1$ 的字符数组

$pattern \leftarrow s_2$ 的字符数组

while `find_substring(data, pattern)` 能找到 pos

从 pos 开始删除 $pattern.len$ 个字符

返回 $data$ 组成的新字符串

函数调用关系可表示为：`main()` -> `delete_substring()` -> `find_substring()` -> 输出删除后的字符串。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

本部分代码位于串 `/ex_4/src/lib.rs`，主程序位于串 `/ex_4/src/main.rs`，测试代码位于串 `/ex_4/tests/test_fn.rs`。主程序演示从 `abcxxabcxxabc` 中删除所有 `abc`。

测试数据覆盖普通多次删除、连续重叠删除和空模式串，重点验证循环删除是否能一直执行到没有 s_2 子串为止。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	$s_1=abcxxabcxxabc$, $s_2=abc$	删除后为 <code>xxxx</code> ；三个 <code>abc</code> 均被删除。
2	$s_1=aaaa$, $s_2=aa$	第一次删去前两个 <code>a</code> 后继续匹配，最终结果为空串。
3	$s_1=abc$, $s_2=$ 空串	直接返回 <code>abc</code> ，避免空模式串导致无限删除。
4	$s_1=abcdef$, $s_2=gh$	找不到子串，结果仍为 <code>abcdef</code> 。

运行 `cargo test` 后，本题 3 个单元测试全部通过；运行 `cargo run` 后，程序输出删除后结果 `xxxx`，说明所有指定子串均已删除。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于每删除一次后都要重新查找，因为字符串内容和后续位置会改变；难点在于处理 `s2` 为空串时必须提前返回。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：查找和删除职责分明，代码容易理解。可以改进的方向包括：不用 `Vec::remove` 多次移动元素，而是构造新字符串或使用更高效的匹配算法减少移动次数。还有哪些实现方法？

还可以从左到右扫描 `s1`，把不属于 `s2` 匹配片段的字符复制到新结果中；也可以直接使用 `replace` 方法把 `s2` 替换为空串，但手写循环更符合本题练习要求。

串进阶练习四

【问题描述】

题目要求采用定长顺序存储结构存储串，编写实现串通配符匹配的函数 `pattern_index()`。其中通配符只有 `?`，它可以和任一字符匹配成功。示例 `pattern_index("?re", "there are")` 返回 2。

本实验在串/ex_5 中实现 `pattern_index(pattern: &str, text: &str) -> Option<usize>`。返回值为匹配成功的起始下标，若不存在匹配则返回 `None`。

【基本思路】

算法把模式串和目标串都转为字符数组。如果模式串为空，则认为从下标 0 即可匹配；如果模式串长度大于目标串，则直接返回 `None`。否则枚举目标串中每一个可能起始位置。

对于某个起始位置 `start`，逐个比较 `pattern[i]` 与 `text[start+i]`。当 `pattern[i]` 为 `?` 时直接认为匹配；否则要求两个字符相等。若整段都匹配，则返回 `start`。

`pattern_index(pattern, text)`

`p <- pattern` 的字符数组，`t <- text` 的字符数组

 if `p` 为空 return `Some(0)`

 if `p.len > t.len` return `None`

 for `start` from 0 to `t.len-p.len`

 若 `p` 中每个字符都能与 `t[start..]` 对应匹配

 return `Some(start)`

 return `None`

函数调用关系可表示为：`main() -> pattern_index() -> pattern_matches() -> 输出匹配起始位置`。

【实验结果及分析】

1. 实验结果：

1.1 实验过程描述

本部分代码位于串/ex_5/src/lib.rs，主程序位于串/ex_5/src/main.rs，测试代码位于串/ex_5/tests/test_fn.rs。主程序使用题目示例 `?re` 与 `there are`。

测试数据覆盖题目示例、首部匹配、无法匹配、模式串过长以及空模式串，重点验证 `?` 是否只匹配一个任意字符。

1.2 测试结果

编号	测试数据	预期结果 / 实际分析
1	<code>pattern=?re, text=there are</code>	返回 <code>Some(2)</code> ； <code>?</code> 匹配 <code>h</code> , <code>r</code> 和 <code>e</code> 分别匹配成功。
2	<code>pattern=t?e, text=there are</code>	返回 <code>Some(0)</code> ； <code>?</code> 匹配 <code>h</code> 。
3	<code>pattern=a?z, text=there are</code>	返回 <code>None</code> ；不存在满足末尾 <code>z</code> 的匹配段。
4	<code>pattern=too long, text=short</code>	返回 <code>None</code> ；模式串长度大于目标串。

运行 `cargo test` 后，本题 3 个单元测试全部通过；运行 `cargo run` 后，程序输出“匹配成功，起始位置: 2”，与题目示例一致。

2. 结果及方案分析

2.1 本实验的重点及难点

本实验的重点在于把 `?` 处理为“匹配任一单个字符”，而不是匹配任意长度字符串；难点在于枚举起始位置时要保证剩余长度足够。

2.2 本实验设计的优点及可以改进的方向

本实验设计的优点是：匹配规则简单清楚，返回 `Option<usize>` 能区分匹配成功与失败。可以改进的方向包括：扩展 `*` 等更复杂通配符，或返回所有匹配位置。还有哪些实现方法？

还可以把模式串转换为正则表达式后调用库函数匹配；也可以使用朴素子串匹配算法稍作修改，使 `?` 在比较阶段跳过相等性判断。当前实现即采用后一种思路。